

Aula 4 - Entidades, Atributos e Objetivos de Medição

Prof. Danilo de Quadros Maia Filho

Departamento:
Engenharia de Software

1 Objetivos da Aula

2 Entidades

- Tipos de Entidades
- Atributos

3 Aspectos de Medição

- Medição Direta ou Derivada
- Escalas e Representações

4 Goal-Question-Metric Framework

- GQM: Contexto e Uso
- GQM: Template

- 1 Objetivos da Aula
- 2 Entidades
 - Tipos de Entidades
 - Atributos
- 3 Aspectos de Medição
 - Medição Direta ou Derivada
 - Escalas e Representações
- 4 Goal-Question-Metric Framework
 - GQM: Contexto e Uso
 - GQM: Template

O que vamos aprender hoje:

- Definir formalmente:
 - Entidades e Atributos
 - Características de Atributos
 - Escalas e Representações
 - Framework para aplicação de Medição
 - Método para definição sistematizada de objetivos

1 Objetivos da Aula

2 Entidades

- Tipos de Entidades
- Atributos

3 Aspectos de Medição

- Medição Direta ou Derivada
- Escalas e Representações

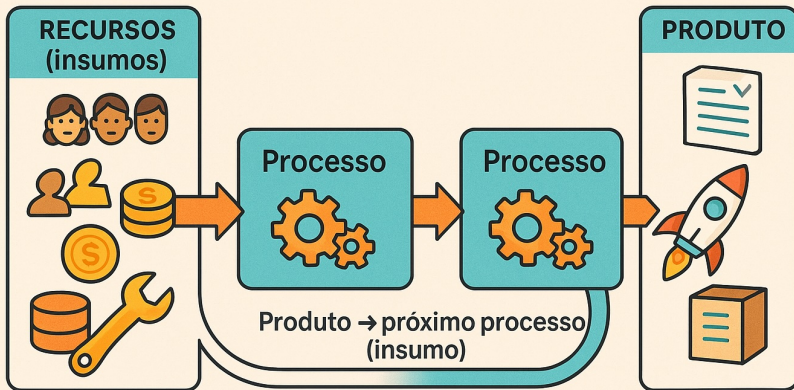
4 Goal-Question-Metric Framework

- GQM: Contexto e Uso
- GQM: Template

As entidades de software podem ser classificadas como:

- **Processes:** Software-related activities
- **Products:** Artifacts, deliverables, or documents that result from a process activity
- **Resources:** Entities required by a process activity

Processos, Recursos e Produtos – Quando a saída vira insumo



Produtos podem ser consumidos como recurso por processos seguintes (cadeia iterativa)

Figura 1: Processos, Produtos e Recursos

A process is usually associated with some time scale. Process activities have duration—they occur over time, and they may be ordered or related in some way that depends on time, so that one activity must be completed before another can begin. The timing can be explicit, as when design must be complete by October 31, or implicit, as when a flow diagram shows that design must be completed before coding can begin.

Examples: development process, maintenance process, the reuse process, the configuration management process, the testing process, etc.

Resources and products are associated with a process. Each process activity uses resources and products, and produces products. Thus, the product of one activity may feed another activity. For example, a design document can be the product of the design activity, which is then used as an input to the coding activity.

- **Internal attributes:** Attributes that can be measured purely in terms of the product, process, or resource itself. An internal attribute can be measured by examining the product, process, or resource on its own, without considering its behavior.
- **External attributes:** Attributes that can be measured only with respect to how the product, process, or resource relates to its environment. Here, the behavior of the process, product, or resource is important, rather than the entity itself.

INTERNAL vs EXTERNAL SOFTWARE ATTRIBUTES

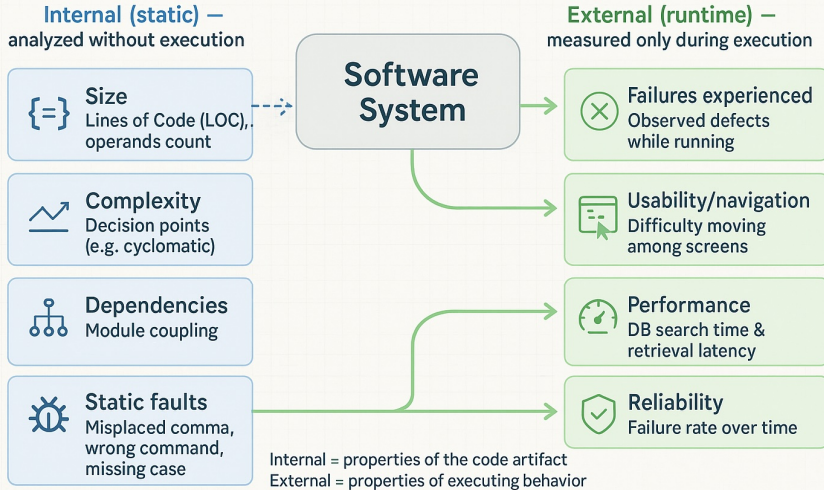


Figura 2: Atributos de um Software

Stakeholders & Software Attributes (External vs Internal)

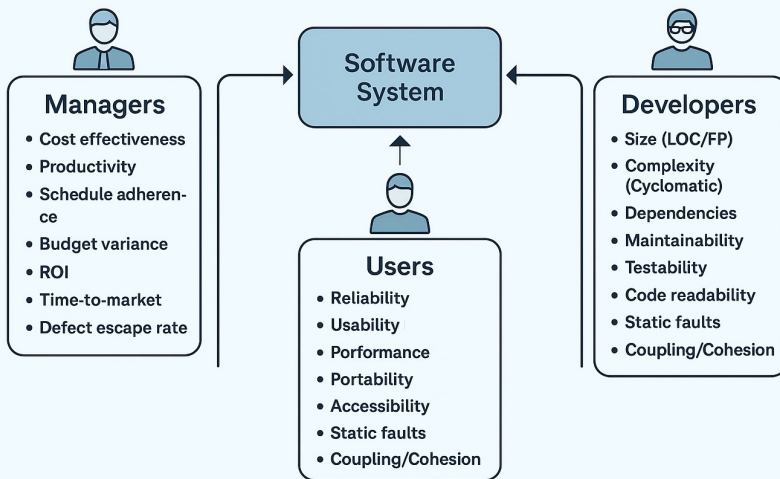


Figura 3: Relação entre atributos e stakeholders

Entities	Attributes	
	Internal	External
Products		
Requirements Use case models and scenarios	Size, reuse, modularity, redundancy, functionality, syntactic correctness, etc.	Comprehensibility, maintainability, etc.
Designs, Design models	Size, reuse, modularity, coupling, cohesiveness, functionality, etc.	Quality, complexity, maintainability, etc.
Code	Size, reuse, modularity, coupling, functionality, algorithmic complexity, control-flow structuredness, etc.	Reliability, usability, maintainability, etc.
Test requirements	Size, etc.	Effectiveness, etc.
Test data	Size, coverage,	Fault-finding ability
Test harness	Languages supported, features	Ease of use
...	...	...

Figura 4: Componentes de Medição de Software - Produto

...	...	...
<i>Processes</i>		
Constructing requirements	Time, effort, number of requirements changes, etc.	Quality, cost, stability, etc.
Detailed design	Time, effort, number of specification faults found, etc.	Cost, cost-effectiveness, etc.
Testing	Time, effort, number of coding faults found, etc.	Cost, cost-effectiveness, stability, etc.
...	...	...

Figura 5: Componentes de Medição de Software - Processo

...	...	...
<i>Resources</i>		
Personnel	Age, price, etc.	Productivity, experience, intelligence, etc.
Teams	Size, communication level, structuredness, etc.	Productivity, quality, etc.
Software	Price, size, etc.	Usability, reliability, etc.
Hardware	Price, speed, memory size, etc.	Reliability, etc.
Offices	Size, temperature, light, etc.	Comfort, quality, etc.
...	...	...

Figura 6: Componentes de Medição de Software - Recurso

- 1 Objetivos da Aula
- 2 Entidades
 - Tipos de Entidades
 - Atributos
- 3 Aspectos de Medição
 - Medição Direta ou Derivada
 - Escalas e Representações
- 4 Goal-Question-Metric Framework
 - GQM: Contexto e Uso
 - GQM: Template

Uma vez que temos um modelo das entidades e atributos envolvidos, podemos definir a medida em termos deles. Muitos dos exemplos que utilizamos empregam mapeamentos diretos do atributo para o número, e usamos o número para responder perguntas ou avaliar situações. Mas, quando existem relações complexas entre os atributos, ou quando um atributo precisa ser medido pela combinação de vários de seus aspectos, então precisamos de um modelo de como combinar as medidas relacionadas. É por essa razão que distinguimos medição direta de medição derivada.

Medição Direta: medição direta de um atributo de uma entidade não envolve outro atributo ou entidade.

- Size of source code (measured by LOC)
- Schedule of the testing process (measured by elapsed time in hours)
- Number of defects discovered (measured by counting defects)
- Time a programmer spends on a project (measured by months worked)

Medição Derivada: medição calculada a partir de vários atributos de entidades.

$$\textit{densidade} = \textit{massa} / \textit{volume} \quad (1)$$

Programmer productivity	LOC produced/person-months of effort
Module defect density	Number of defects/module size
Defect detection efficiency	Number of defects detected/total number of defects
Requirements stability	Number of initial requirements/total number of requirements
Test coverage	Number of test requirements covered/total number of test requirements
System spoilage	Effort spent fixing faults/total project effort

Tabela 1: Exemplos de Medições Derivadas

Faults in subsystems

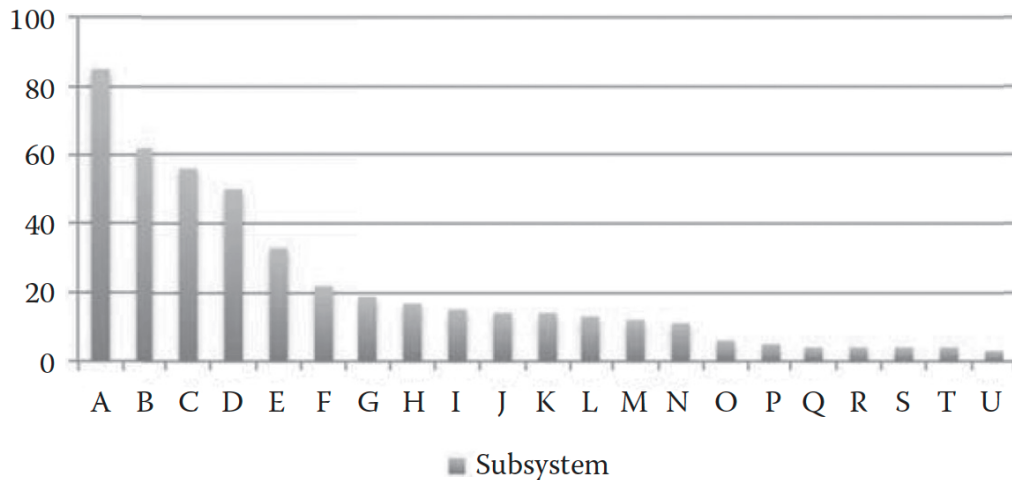


Figura 7: Exemplo de Faltas por subsistema em um grande sistema

Faults per KLOC in subsystems

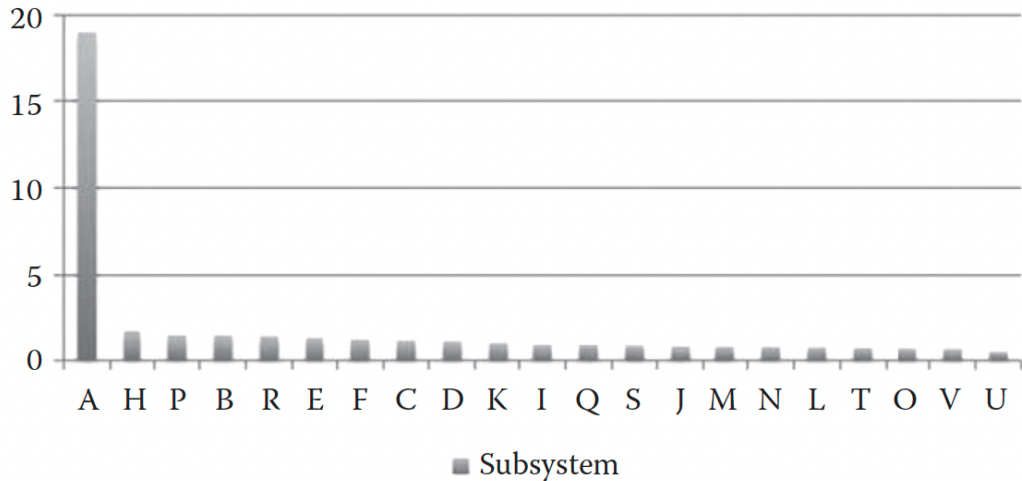


Figura 8: Densidade de faltas por subsistema - por Faltas/KLOC

EXAMPLE 2.12

A common generic model for predicting the effort required in software projects has the form

$$E = aS^b$$

where E is effort in person-months, S is the size (in LOC) of the system to be constructed, and a and b are constants. We will examine many of these models in Chapter 11.

Figura 9: Exemplo de modelo para cálculo de esforço em um projeto

EXAMPLE 2.14

A well-known reliability model is based on an exponential distribution for the time to the i th failure of the product. This distribution is described by the formula

$$F(t) = 1 - e^{-(N-i+1)at}$$

Here, N represents the number of faults initially residing in the program, while a represents the overall rate of occurrence of failures. There are many ways that the model parameters N and a can be estimated, including sophisticated techniques such as maximum likelihood estimation. The details of these prediction systems will be discussed in Chapter 11.

Figura 10: Exemplo de modelo para cálculo da distribuição de probabilidade para a i ésima Falha do produto num dado tempo t

- 1 Como determinamos quando um sistema de relações numéricas é preferível a outro?
- 2 Como sabemos se um determinado sistema de relações empíricas possui uma representação em um dado sistema de relações numéricas?
- 3 O que fazemos quando temos várias representações possíveis (e, portanto, muitas escalas) no mesmo sistema de relações numéricas?

Classificamos as medições em 5 principais categorias de escalas (existem outras):

- 1 Nominal
- 2 Ordinal
- 3 Intervalar
- 4 Razão
- 5 Absoluto

Admissible Transformation

É qualquer transformação aplicada aos valores numéricos de uma medida que preserva a estrutura empírica do atributo medido.

Intuitivamente: você mede algo no mundo real (ordens, igualdade, proporções...) e mapeia para números. Se aplicar uma função f a todos os números e tudo que é observável continua verdadeiro, então f é admissível para aquela escala.

Tipo de escala	Tranformações Admissíveis	O que a transformação preserva	Exemplos de métricas
Nominal	Any bijection (re-labeling)	Equality / difference of labels	Defect category, requirement type
Ordinal	Any strictly increasing function	Order (ranks)	Likert 1–5, severity (Low/High)
Interval	$f(x) = ax + b, a > 0$	Differences (intervals), not ratios	$^{\circ}\text{C} \leftrightarrow ^{\circ}\text{F}$, standardized scores (z)
Ratio	$f(x) = ax, a > 0$	Ratios and differences; fixed zero	defect density, team velocity
Absolute	Identity $f(x) = x$	All relations (no freedom)	Pure counts (number of failures), Response time (s), LOC, cost (\$)

Tabela 2: Outras Datas

• Nominal

- Defect category (UI, Logic, Performance, ...). *Transformação admissível*: renomear rótulos por qualquer bijeção (ex.: $UI \mapsto A$, $Logic \mapsto B$).
- Requirement type (Functional, Nonfunctional, Security, ...). *Transformação admissível*: recodificação 1-para-1 dos rótulos.

• Ordinal

- Bug severity (Low < Medium < High < Critical). *Transformação admissível*: mapear para qualquer sequência estritamente crescente (ex.: $\{1, 2, 3, 4\} \rightarrow \{10, 20, 30, 50\}$).
- Issue priority ($P4 < P3 < P2 < P1$). *Transformação admissível*: $f(n) = 2n + 1$ (monótona crescente sobre os ranks).

• Interval

- Build server temperature in $^{\circ}\text{C}$. *Transformação admissível*: $f(x) = \frac{9}{5}x + 32$ (conversão para $^{\circ}\text{F}$).
- Release time coded as days since project start. *Transformação admissível*: $f(x) = ax + b$ com $a > 0$ (troca de unidade e da origem temporal).

• Ratio

- API response time (milliseconds). *Transformação admissível*: $f(x) = x/1000$ (converter para segundos).
- Defect density (defects/KLOC). *Transformação admissível*: $f(x) = 100x$ (defeitos por 100 KLOC) ou $f(x) = x/1000$ (defeitos por LOC).

• Absolute

- Number of failing tests (count). *Transformação admissível*: identidade, $f(x) = x$.
- Number of modules/components (count). *Transformação admissível*: identidade, $f(x) = x$.

- 1 Objetivos da Aula
- 2 Entidades
 - Tipos de Entidades
 - Atributos
- 3 Aspectos de Medição
 - Medição Direta ou Derivada
 - Escalas e Representações
- 4 Goal-Question-Metric Framework
 - GQM: Contexto e Uso
 - GQM: Template

Erro comum em Engenharia de Software:

Muitos programas de métricas começam medindo o que é conveniente ou fácil de medir, em vez de medir o que é necessário.

Esses programas frequentemente fracassam porque os dados resultantes não são úteis para os desenvolvedores e mantenedores do software. Um programa de métricas pode ter mais sucesso se for concebido tendo em mente os objetivos do projeto.

Definição de GQM:

A abordagem *Goal-Question-Metric* (GQM) para seleção de métricas, sugerida inicialmente por Basili e seus colegas, é uma abordagem eficaz (Basili and Weiss 1984; Basili and Rombach 1988; Basili et al. 2009).

Para usar GQM, primeiro identificam-se os objetivos gerais da organização. Dependendo da situação, a organização pode ser uma corporação inteira, um departamento ou laboratório individual, ou um único projeto ou grupo. Em seguida, geram-se perguntas cujas respostas são necessárias para determinar se os objetivos estão sendo atendidos. Por fim, analisa-se cada pergunta a fim de identificar as medições de que se necessita para respondê-la.

A abordagem GQM fornece um arcabouço composto por três etapas:

- 1 Liste os principais objetivos do projeto de desenvolvimento ou de manutenção.
- 2 A partir de cada objetivo, derive as perguntas que precisam ser respondidas para determinar se os objetivos estão sendo atingidos.
- 3 Decida o que deve ser medido para que seja possível responder adequadamente às perguntas.

Goal-Question-Metric

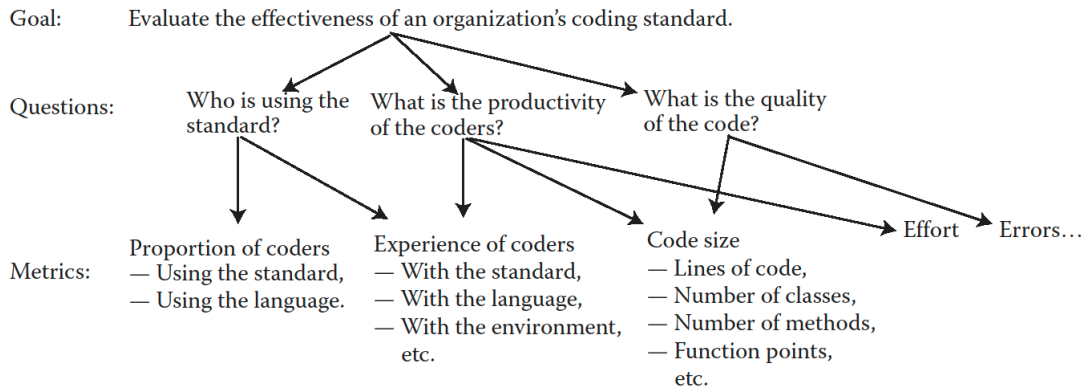


Figura 11: Derivação de métricas a partir dos objetivos

EXAMPLE 3.8

AT&T used GQM to help determine which metrics were appropriate for assessing their inspection process (Barnard and Price 1994). Their goals, with the questions and metrics derived, are shown in Table 3.3.

Figura 12: GQM: Exemplo de derivação

Goal	Questions	Metrics
Plan	How much does the inspection process cost?	Average effort per KLOC Percentage of reinspections
	How much calendar time does the inspection process take?	Average effort per KLOC Total KLOC inspected
Monitor and control	What is the quality of the inspected software?	Average faults detected per KLOC Average inspection rate Average preparation rate
	To what degree did the staff conform to the procedures?	Average inspection rate Average preparation rate Average lines of code inspected Percentage of reinspections
Improve	What is the status of the inspection process?	Total KLOC inspected
	How effective is the inspection process?	Defect removal efficiency Average faults detected per KLOC Average inspection rate Average preparation rate Average lines of code inspected
	What is the productivity of the inspection process?	Average effort per fault detected Average inspection rate Average preparation rate Average lines of code inspected

Figura 13: GQM: Exemplo de derivação

Para auxiliar na geração de objetivos, perguntas e métricas, Basili e Rombach forneceram uma série de modelos.

Modelos para definição de objetivos:

Purpose (Propósito): *Para* (caracterizar, avaliar, prever, motivar, etc.) o (processo, produto, modelo, métrica, etc.) *a fim de* (compreender, avaliar, gerenciar, projetar, aprender, melhorar, etc.) o objeto em questão.

Exemplo: Avaliar o processo de manutenção a fim de melhorá-lo.

Perspective (Perspectiva): Examinar o (custo, efetividade, correção, defeitos, mudanças, medidas de produto, etc.) do ponto de vista do (desenvolvedor, gerente, cliente, etc.).

Exemplo: Examinar o custo do ponto de vista do gerente.

Environment (Ambiente): O ambiente consiste nos seguintes elementos: fatores de processo, fatores de pessoas, fatores do problema, métodos, ferramentas, restrições, etc.

Exemplo: A equipe de manutenção é composta por programadores pouco motivados que têm acesso limitado a ferramentas.

EXAMPLE 3.10

Fuggetta et al. report on the use of GQM on a software process at the Digital Software Engineering Center in Gallarate, Italy (Fuggetta et al. 1998). The group identified five goals including the following goal, which has a format consistent with our templates:

Analyze the design and qualification phases of the development process for the purpose of evaluating failure detection effectiveness from the viewpoint of the management and the development team.

The measurement group derived 35 questions that could be answered using data from approximately 50 metrics.

Figura 14: GQM: Exemplo de como definir uma métrica

- ❶ FENTON, Norman; BIEMAN, James. Software metrics: a rigorous and practical approach. CRC press, 2014.
- ❷ WOHLIN, Claes et al. Experimentation in software engineering. Berlin: Springer, 2012.
- ❸ JURISTO, Natalia; MORENO, Ana M. Basics of software engineering experimentation. Springer Science & Business Media, 2013.
- ❹ EMPIRICAL SOFTWARE ENGINEERING: an international journal. Boston: Kluwer Academic Publishers, 1996-. j20. ISSN 1573-7616. Disponível em: <https://link.springer.com/journal/10664>. Acesso em: 22 jul. 2024. (Periódico On-line).
- ❺ PRESSMAN, Roger S.; MAXIM, Bruce R.. Engenharia de software: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021. E-book. ISBN 9786558040118. (Livro Eletrônico).
- ❻ WOHLIN, Claes et al. Experimentation in software engineering. New York: Springer, c2012. xxiii, 236 p. ISBN 9783642432262. (Disponível no Acervo).

- 1 JURISTO, Natalia. Basics of Software Engineering Experimentation. 1st ed. 2001. XXII, 396 p ISBN 9781475733044. (Livro Eletrônico).
- 2 LAIRD, Linda M.; BRENNAN, M. Carol,. Software measurement and estimation: a practical approach. Hoboken, N.J.: Wiley-Interscience, 2006. 1 online resource (xvi, 257 ISBN 0471792535 (electronic bk.) Disponível em : <https://ieeexplore.ieee.org/book/5988896>. Acesso em: 22 jul. 2024. (Livro Eletrônico).
- 3 SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson Education do Brasil, c2019. xii, 756 p. ISBN 9788543024974. (Disponível no Acervo).
- 4 DEVORE, Jay L. Probabilidade e estatística para engenharia e ciências. 3. São Paulo Cengage Learning 2018 1 recurso online ISBN 9788522128044. (Livro Eletrônico).
- 5 IEEE TRANSACTIONS ON SOFTWARE ENGINEERING. New York: IEEE Computer Society ,1975-. Mensal,. ISSN 0098-5589. Disponível em: <https://ieeexplore.ieee.org/xpl/RecentIssue.jsp?punumber=32>. Acesso em: 22 jul. 2024. (Periódico On-line).
- 6 JONES, Capers. Applied Software Measurement, 3rd edition. 2008. 1 online resource (696 pages). (Livro Eletrônico).